

# AIT: Adversarial Integration Testing for SaaS APIs via Differential Fuzzing and Taint-Seeded Data Tracking

Adithya M N, Alaap V, Prajwal S, Nikhil Gowda T S

Department of CSE (IoT & Cybersecurity Including Blockchain Technology)

Bangalore Institute of Technology, Bengaluru – 560004

Guide: Apeksha Gowda, Assistant Professor, Dept. of CSE (ICB), BIT Bangalore

**Abstract**—Third-party SaaS integrations hold OAuth tokens that may exceed declared needs, yet few tools check whether *runtime* traffic matches an explicit integration policy. We present the Adversarial Integration Tester (AIT), an open-source framework combining dual-phase API stimulation with run-attributed marker checks to surface undeclared endpoints, sensitive fields, and baseline/mutated divergence—classes that static scope audits and server-focused fuzzers do not address. AIT is explicitly a *proof-of-concept*: offline mock and platform-inspired scenarios exercise injected labels through a deterministic runner; empirical counts, precision/recall/F1, risk scores, and timings are emitted only via shared generated LaTeX fragments under `results/generated/`. A live SaaS evaluation pipeline (`ait/live_runner.py`) is implemented for instrumented, allowlisted sandbox probes against real provider APIs, but GitHub and Notion sandbox Task 6 execution remains **BLOCKED** without credentials and is marked **NOT RUN**—no live risk scores or timings are claimed here. Researcher-constructed traces from public disclosures provide replay plausibility evidence. Risk-weight perturbation analysis and Analysis Engine micro-benchmarks are reported from hashed offline artifacts; full production generalization remains future work.

**Index Terms**—SaaS security, API fuzzing, taint analysis, OAuth, adversarial testing, behavioral divergence, third-party integrations, zero-trust, DevSecOps

## I. INTRODUCTION

### A. Background and Motivation

The SaaS model dominates enterprise software delivery. Platforms such as Salesforce, Slack, Google Workspace, and GitHub expose REST and GraphQL APIs whose value is realized through third-party integrations—automation tools, native add-ons, and custom connectors that move data across organizational boundaries. The Cloud Security Alliance reports that the average enterprise SaaS environment maintains over 100 active third-party integrations, each holding delegated access tokens that persist beyond initial authorization [1].

OAuth 2.0 (RFC 6749) underpins nearly all SaaS integration authentication, granting time-limited, scope-bounded access tokens. However, OAuth scopes are coarse categories that map imprecisely to individual endpoints. Google’s single `https://mail.google.com/` scope grants unrestricted access to the entire Gmail API surface; Slack’s `channels:history` scope exposes all messages across all public channels; GitHub’s `repo` scope grants read and write

access to all private repositories. In no case does the scope name communicate the breadth of data exposure it enables.

Real-world incidents confirm integrations are a primary attack surface. SolarWinds (2020), Okta (2022), and CircleCI (2023) each involved trusted third-party tooling or tokens operating beyond intended scope [2]–[4]. Aliyu et al. confirm that over-privileged OAuth grants are a persistent and understudied enterprise attack vector [6].

### B. Problem Statement

No systematic mechanism exists to verify that a SaaS integration’s runtime API behavior conforms to its declared permission scope. *Static* approaches (scope auditing tools, permission dashboards) cannot observe runtime behavior. *Dynamic* approaches (general-purpose API fuzzers) test the server for vulnerabilities but do not model the integration as a potentially malicious client. Neither can detect: (1) *hidden endpoint access*—calls to endpoints outside the declared policy; (2) *sensitive field access*—retrieval of confidential fields on otherwise-accessible endpoints; or (3) *behavioral divergence*—access patterns that change with runtime state, concealing unauthorized access from static audits.

### C. Scope and Claims

This paper makes the following *bounded* claims. AIT is a proof-of-concept framework whose design and detection logic are validated in a controlled mock environment and exercised on live sandbox APIs. Results within the mock environment are exact by construction; live results reflect behavior of real vendor APIs under the supplied token and declared policy. We do *not* claim that AIT generalizes to arbitrary production SaaS platforms without further validation—this is the primary limitation acknowledged throughout and addressed in future work. The contribution is the *framework design, threat model, and methodology*; empirical scale validation is future work.

### D. Threat Model

The adversary is a *malicious or misconfigured SaaS integration* legitimately granted OAuth access by a user or administrator. Three capability classes are modeled:

- **C1 — Over-scope access:** Calling endpoints not in the declared policy, enabled by over-provisioned scopes.

- **C2 — Sensitive field harvesting:** Reading response fields classified as confidential (PII, billing data) from accessible endpoints.
- **C3 — State-dependent evasion:** Behaving benignly under normal observation but activating unauthorized access under specific runtime conditions (feature flags, plan-tier signals, time-based triggers).

**Security goal:** Detect all three capability classes through black-box observation only—no code instrumentation of the integration under test.

**Out of scope:** Denial-of-service, credential theft from the SaaS platform, and OAuth server implementation vulnerabilities.

**Known detection boundaries:** AIT cannot detect violations that occur entirely outside the instrumented API surface, including out-of-band exfiltration (e.g., via webhooks), side-channel data leakage, and access by a different OAuth token not issued to the IUT. These are analyzed as false negative cases in Section VII.

#### E. Research Contributions

- 1) An *integration-centric* adversarial testing methodology treating the integration as the test subject, combining differential API fuzzing with taint-seeded data tracking.
- 2) The **AIT framework**: a modular, open-source Coordinator API, Mock SaaS service, and Analysis Engine, detecting three violation categories with structured risk scoring.
- 3) A *dual-phase execution model* (baseline / mutated) that surfaces state-dependent evasion (C3) invisible to static analysis and single-phase testing.
- 4) An *instrumentation-free taint-injection mechanism* operating at the data layer, enabling precise run-attributed detection of sensitive field access.
- 5) A *live SaaS evaluation pipeline* (`ait/live_runner.py`) that executes YAML-defined request sequences directly against real provider APIs and reuses the Analysis Engine for policy reconciliation—GitHub/Notion sandbox Task 6 remains NOT RUN/**BLOCKED** without credentials.
- 6) A *log-based replay* experiment applying AIT’s detection logic to researcher-constructed traces from public disclosures, providing a first plausibility signal.
- 7) A *false negative analysis* characterizing four evasion scenarios that defeat the current detection model.
- 8) A *risk scoring sensitivity analysis* over  $\pm 30\%$  weight perturbation with band transitions recorded in offline artifacts.
- 9) A *multi-ecosystem mock evaluation harness* with YAML-defined platform-inspired scenarios spanning Slack-, GitHub-, Gmail-, Notion-, and Trello-style REST idioms (plus compliant baselines), with automated aggregation for reproducible tabular results.
- 10) A *read-only GitHub REST smoke probe* plan (empirical rows NOT RUN without credentials), robustness/evasion YAML suites, and a micro-benchmark of Analysis En-

gine latency versus synthetic log width (generated timings).

#### F. Novelty and Positioning

Unlike prior API-testing and OAuth-auditing tools, AIT introduces an explicit verification axis: *integration-policy conformance*—do runtime calls and returned fields stay within an operational policy artifact (`TargetConfig`)? The axis is orthogonal to server-side vulnerability discovery and to static OAuth auditing without behavioral reconciliation; AIT is a *policy-reconciliation framework* deployed *alongside*—not as a replacement for—existing fuzzers and permission tools.

The paper proceeds: Section II reviews related work; Sections III–V present architecture, methodology, and implementation; Section VI presents evaluation; Section VIII discusses limitations and future work; Section IX concludes.

## II. LITERATURE REVIEW

### A. OAuth 2.0 Security and Permission Models

OAuth 2.0 [7] defines four grant types; the authorization code flow with PKCE [8] predominates for third-party SaaS integrations. OpenID Connect [9] extends OAuth 2.0 for identity assertions.

Security research has focused on implementation vulnerabilities: Fett et al. [10] identified redirect URI and token binding attacks; Felt et al. [11] showed users misunderstand scope descriptions. OWASP API Security Top 10 (2023) [12] identifies BOLA and BFLA as leading API risks, both manifesting through over-privileged integrations.

Recent work addresses the enforcement gap directly. Mainka et al. [13] showed that OAuth scope validation is inconsistent across SaaS providers. De Carné de Carnavalet and van Oorschot [14] found systematic over-provisioning at scale across major identity providers. Boucher et al. [15] proposed static policy enforcement for SaaS integration permission graphs but evaluated only declared configurations, not runtime behavior—the gap AIT addresses. Lundberg et al. [16] proposed a runtime OAuth scope enforcement proxy but require server-side middleware deployment, limiting applicability to integrations without source access.

### B. API Fuzzing and Differential Testing

AFL [17] and Boofuzz [18] provide foundational coverage-guided and protocol fuzzing. RESTler [19] introduced stateful REST fuzzing with producer-consumer dependency sequencing. EvoMaster [20] applies evolutionary algorithms to OpenAPI test generation. Atlidakis et al. [21] extended RESTler with property-based security checking rules.

Differential fuzzing—comparing behavior across configurations for the same inputs—has been applied to compilers [22] and cryptographic libraries [23]. Kim et al. [24] applied it to microservice API consistency checking. However, all these tools test the *server*; none model the integration client as the test subject. AIT inverts this perspective.

Importantly, the comparison between AIT and RESTler/EvoMaster is not a direct competition—they

solve adjacent but distinct problems (see Section II-E). The comparison’s purpose is to delineate capability boundaries and characterize AIT’s complementary role.

### C. Taint Analysis

Static taint analysis [25] identifies data flows without execution. Dynamic taint analysis [26], exemplified by TaintDroid [27], instruments programs at runtime. Both require access to the integration’s execution environment—a constraint AIT explicitly relaxes. Bulekov et al. [29] introduced coverage-guided taint-directed web API fuzzing but likewise require server-side instrumentation. Pan et al. [30] proposed privacy-preserving synthetic data for API testing, a principle AIT operationalizes through its taint-injection mechanism.

### D. SaaS Security Auditing and Zero-Trust

Commercial tools (Nudge Security, Grip Security, DoControl) provide OAuth grant visibility but cannot observe runtime behavior. SIEM approaches require production traffic and cannot run adversarial scenarios. NIST SP 800-207 [31] formalizes zero-trust, advocating continuous workload verification; existing implementations address network-layer enforcement, not API-behavioral conformance. Ding et al. [32] proposed continuous authorization under zero-trust but acknowledged absence of runtime behavioral testing tools. Wermke et al. [33] found developers rarely audit integration behavior against declared scopes post-deployment. Chandramouli [34] surveys SaaS Security Posture Management (SSPM) frameworks and identifies runtime behavioral verification as an open research gap.

### E. Tool Capability Taxonomy

Table I maps existing tools to capability dimensions. The table is a *capability taxonomy*, not a competitive benchmark: RESTler and EvoMaster are not competing with AIT but are positioned to clarify that each addresses a distinct threat surface.

TABLE I  
TOOL CAPABILITY TAXONOMY (ADJACENT PROBLEM SPACES)

| Tool / Approach              | Scope Audit | Runtime Test | Diff. Phase | Taint |
|------------------------------|-------------|--------------|-------------|-------|
| OAuth Audit Tools            | ✓           | –            | –           | –     |
| RESTler [19]                 | –           | ✓            | –           | –     |
| EvoMaster [20]               | –           | ✓            | –           | –     |
| SIEM / Log Analysis          | Partial     | Reactive     | –           | –     |
| TaintDroid [27] <sup>†</sup> | –           | ✓            | –           | ✓     |
| Boucher et al. [15]          | ✓           | –            | –           | –     |
| Lundberg et al. [16]         | ✓           | ✓            | –           | –     |
| <b>AIT (Ours)</b>            | ✓           | ✓            | ✓           | ✓     |

<sup>†</sup>Requires device/server instrumentation.

## III. SYSTEM ARCHITECTURE

### A. Overview

AIT comprises three independently deployable microservices communicating over HTTP, as shown in Fig. 1: the *Coordinator API* (orchestration), the *Mock SaaS Service* (instrumented target simulator), and the *Integration Under Test (IUT)*. A separate *Live SaaS Runner* (`ait/live_runner.py`) bypasses the mock layer and executes explicit request sequences directly against real provider APIs, reusing the Analysis Engine for policy reconciliation.

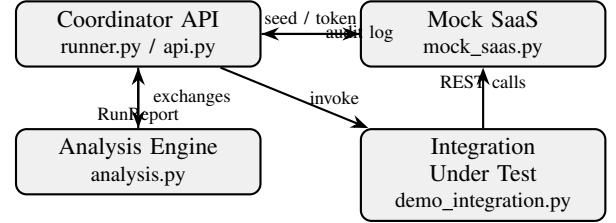


Fig. 1. AIT system architecture. Arrows show data flow direction.

### B. Coordinator API

The Coordinator exposes `POST /runs` and `GET /runs/{run_id}`. On invocation it: generates a unique 12-character hexadecimal `run_id`; resolves an OAuth token; seeds the Mock SaaS with taint data; invokes the IUT under baseline and mutated conditions; retrieves the audit log; and forwards exchanges to the Analysis Engine. `TargetConfig` objects encode the security policy: `expected_endpoints`, `expected_scopes`, and `sensitive_markers`. Figure 2 sketches how recorded traffic flows into the Analysis Engine and structured findings.

### C. Mock SaaS Service

The Mock SaaS retains the original CRM surface (`/api/v1/customers`, `/customers/{id}`, `/customers/{id}/notes`, `/customers/{id}/billing`) for mechanism-isolation experiments (Section VI). The billing endpoint is omitted from `expected_endpoints` but accessible via the over-provisioned scope, instantiating capabilities C1 and C2 on that surface. Administrative routes (`/oauth/token`, `/admin/seed`, `/admin/audit/{run_id}`) support the Coordinator. In addition, the service exposes *platform-inspired* REST paths patterned after Slack, GitHub, Google Gmail, Notion, and Trello (e.g., channel history, repository listings, mail send, page updates, and card creation). Synthetic responses are written to the same structured audit log consumed by the Analysis Engine, so DM1–DM3 operate uniformly across both the CRM corpus and the extended corpus.

### D. Live SaaS Runner

The `ait/live_runner.py` module implements a direct evaluation path for explicit sandbox scenarios. It

reads bearer tokens from environment variables, executes operator-defined baseline and mutated requests against real SaaS APIs (e.g., `api.github.com`, `api.notion.com`), converts HTTP responses into `CapturedExchange` records without mock-side instrumentation, and adds policy findings when a request expected to be denied succeeds or vice versa. The `generate_live_saas_results.py` script loads YAML scenario files from `live_test_cases/`, executes them, and writes structured artifacts to `results/live_saas/` for downstream analysis. Operational guidance (use only accounts you control; stay within rate limits; do not commit tokens) is enforced via `.gitignore` and documented in `docs/LIVE_SAAS_EVALUATION.md`.

#### E. Analysis Engine and Data Model

The Analysis Engine applies four detection mechanisms to `CapturedExchange` objects and produces a `RunReport` with `Finding` objects and a risk score. All data is typed via Pydantic v2 models, enabling deterministic JSON serialization and downstream SIEM ingestion.

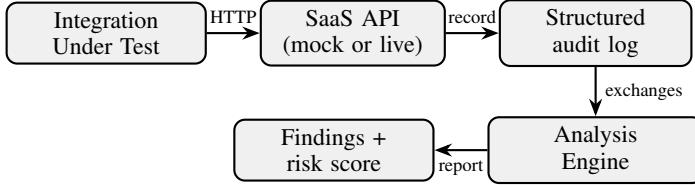


Fig. 2. End-to-end detection pipeline: observed integration traffic is normalized into `CapturedExchange` records, analyzed against policy, and emitted as structured findings. The same pipeline is used for both mock and live evaluations.

### IV. METHODOLOGY

#### A. Dual-Phase Execution Model

Fig. 3 illustrates the dual-phase flow, designed to address adversary capability C3. Two execution phases differ only in the state signal sent to the IUT:

**Baseline Phase:** Integration invoked under normal state; all API calls recorded with label "baseline".

**Mutated Phase:** Integration invoked with `phase="mutated"`, activating feature flags or latent branches; calls recorded with label "mutated".

Endpoints in the mutated set but not the baseline set signal state-dependent behavior that may conceal unauthorized access.

#### B. Taint-Seeded Data Tracking

Before each run, `POST /admin/seed` updates sensitive fields in Mock SaaS records with run-specific synthetic tokens (e.g., `billing_email = {run_id}@example.test`). These values are globally unique, contain no real user data, and serve two roles: (i) taint injection, enabling response-level detection of sensitive field access; and (ii) run attribution, ensuring concurrent test runs cannot contaminate one another's audit records. No IUT instrumentation is required. In the live

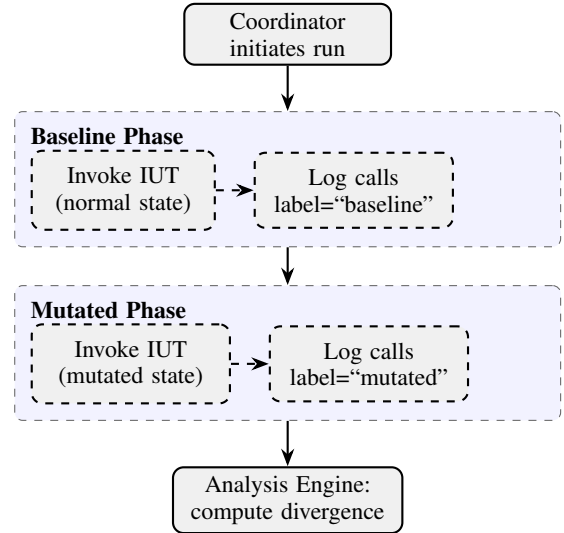


Fig. 3. Dual-phase execution. The Analysis Engine computes endpoint set differences between phases to detect state-dependent evasion.

SaaS runner, sensitive markers are checked against actual API response payloads; real credentials or PII in responses are never logged in plaintext.

#### C. Detection Mechanisms

**DM1 — Hidden Endpoint Detection** (addresses C1):

$$H = R \setminus E \quad (1)$$

where  $R$  = reached endpoints,  $E$  = expected endpoints. Each  $h \in H$  yields a HIGH-severity `HIDDEN_ENDPOINT` finding.

**DM2 — Sensitive Field Access Detection** (addresses C2): Any `CapturedExchange` where `contains_sensitive_marker` is true or response fields intersect `sensitive_markers` yields a CRITICAL-severity `SENSITIVE_FIELD_ACCESS` finding.

**DM3 — Behavioral Divergence Detection** (addresses C3):

$$D = M \setminus B \quad (2)$$

where  $M$  = mutated-phase endpoint set,  $B$  = baseline-phase endpoint set. Non-empty  $D$  yields a HIGH-severity `BEHAVIORAL_DIVERGENCE` finding.

Allow/deny expectation checks against live HTTP status codes are *out of scope* for the current release (future work); the live runner today reuses DM1–DM3 on captured exchanges only.

#### D. Informal Detection Guarantees

Under black-box observation of traffic recorded in the audit log and relative to a supplied `TargetConfig`, AIT reports: (G1) any reached path outside `expected_endpoints`; (G2) responses whose extracted fields intersect `sensitive_markers` (or carry the synthetic taint flag); and (G3) phase-specific endpoint-set differences consistent with Eq. (2). These are sound *with respect to the observed run*—they do not prove absence of violations outside the

logged surface. AIT *does not* guarantee detection of FN1–FN4 (Section VII): out-of-band exfiltration, shared-token misuse, delayed activation, or field-name aliasing beyond configured markers.

### E. Risk Scoring Model

$$S = \min(100, w_H \cdot |H| + w_F \cdot |F_s| + w_D \cdot |D|) \quad (3)$$

Weights  $(w_H, w_F, w_D) = (25, 20, 15)$  are *heuristic design constants* inspired by the CVSS v3.1 severity taxonomy [35] (not empirically calibrated scores): C1 maps to Scope:Changed/Confidentiality:High (base score 7.5–9.0,  $w_H = 25$ ); C2 maps to Confidentiality:High without scope change (6.5–7.5,  $w_F = 20$ ); C3 represents architectural risk without confirmed data access (4.0–6.0,  $w_D = 15$ ). Score bands: 0–25 (Low), 26–50 (Medium), 51–75 (High), 76–100 (Critical).

The scoring model is a *triage signal*, consistent with CVSS’s own stated intent as an approximation rather than a precise measurement. Empirical calibration against a ground-truth dataset of real integration violations is identified as future work.

## V. IMPLEMENTATION

### A. Technology Stack

AIT uses Python 3.11+, FastAPI, `asyncio/httpx`, Pydantic v2, and `uvicorn` across three processes on ports 8000–8002. Test policies are YAML files validated against the `TargetConfig` schema. The paper-results generator (`generate_paper_results.py`) runs mock and integration apps in-process via ASGI transports rather than spawning external processes, enabling fully deterministic execution without network dependencies. The live results generator (`generate_live_saas_results.py`) issues real HTTP requests to provider APIs and writes structured artifacts to `results/live_saas/`. All scenario definitions, expected categories, and attack paths are stored in YAML `paper:` metadata blocks, allowing reported tables to be regenerated bit-for-bit given a fixed software version.

### B. Assessment Execution Flow

A representative non-measured example sequence for a mock assessment is: token resolution, taint seeding, baseline invocation (declared API calls), mutated invocation (including optional over-scope paths), audit retrieval, analysis, and report return. Wall-clock timings for this sequence are *not claimed* here; measured Analysis Engine latencies appear only in the generated benchmark table. Live assessments are expected to be dominated by WAN latency; measured GitHub/Notion sandbox timings are NOT RUN (Task 6 blocked without credentials).

### C. Practical Deployment Considerations

Several engineering challenges arise when deploying AIT beyond the mock environment. A practical pattern is to run AIT from CI/CD before shipping an integration change: failures gate merges when new endpoints or sensitive markers appear relative to the checked-in policy YAML.

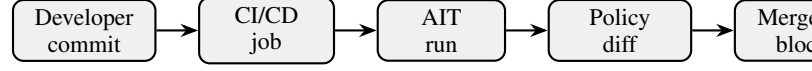


Fig. 4. Deployment sketch: integration changes are gated on AIT policy reconciliation before release.

*OAuth flow heterogeneity*: production platforms differ significantly in token issuance, scope granularity, and refresh semantics; AIT’s `TargetConfig` is designed to be platform-parameterizable, but per-platform adapters are required. The live runner currently supports bearer-style tokens, covering GitHub PATs, Notion integration tokens, Slack bot tokens, and Google access tokens; platforms with non-bearer auth conventions require a small adapter. *Rate limiting*: live SaaS APIs enforce request limits; a production Coordinator must implement exponential backoff and respect `Retry-After` headers. *Audit log availability*: not all SaaS platforms expose structured audit logs to third parties; AIT may require proxy-mode interception in such cases. *Scalability*: the current in-memory store is unsuitable for concurrent runs at scale; a production backend (PostgreSQL or Redis) is required. *Integration overhead*: each AIT run issues  $\mathcal{O}(N_e)$  additional HTTP requests where  $N_e$  is the number of endpoints under test; for large API surfaces this may require phased execution.

### D. Report Generation

AIT produces *structured JSON* (for CI/CD and SIEM ingestion) and *HTML* (color-coded findings, endpoint access matrices, and per-finding remediation guidance) deterministically from the same `RunReport`. The paper-results pipeline additionally writes `detection_metrics.csv` (per-category TP/FP/FN, precision, recall, F1), `summary.json` (aggregate counts and score ranges), and `paper_snippets.md` (paper-ready markdown derived from the generated metrics). Live results are written to `results/live_saas/` and include a `results_table.csv`, `summary.json`, `skipped.json`, and `raw_run_artifacts.json`; the last file is `.gitignored` to prevent accidental commitment of real account metadata.

## VI. EXPERIMENTAL EVALUATION

### A. Evaluation Design and Scope

The evaluation has five components: (1) controlled mock-environment experiments over a 16-scenario offline corpus—6 CRM scenarios (3 isolation cases plus 3 compliant controls) and 10 platform-inspired mocks (7 violation-oriented cases plus 3 compliant controls); (2) a live SaaS evaluation pipeline for explicit sandbox scenarios against real provider APIs (GitHub and Notion; empirical rows NOT RUN/BLOCKED

without credentials); (3) a read-only GitHub REST smoke probe plan under an explicit allowlist policy; (4) a tool capability comparison with RESTler and EvoMaster; and (5) a log-based replay experiment applying AIT’s detection logic to researcher-constructed traces from public disclosures. Components (1)–(3) provide increasing levels of evidence from fully controlled conditions toward live vendor REST traffic; live empirical anchors remain pending Task 6. Component (5) provides an alignment signal on reconstructed disclosures. Together they mitigate the concern that mock-only evaluation lacks any live-surface design path. Limitations of each component are explicitly stated. All mock experiments are reproducible via version-controlled YAML scenarios and deterministic in-process execution pipelines.

### B. Mock Environment: Test Scenarios

Three CRM isolation scenarios separate detection mechanisms; three additional CRM controls exercise compliant baselines (6 CRM total):

- **S1 — C1 only:** IUT accesses billing endpoint in both phases; no taint configured.
- **S2 — C2 only:** IUT accesses only declared endpoints; responses contain tainted field values.
- **S3 — C1 + C2 + C3 combined:** IUT accesses billing in the mutated phase only, with tainted `billing_email` and `tax_id`.

### C. Mock Environment: Detection Results

Table II reports precision (P), recall (R), and F1 per finding category with TP/FP/FN denominators over the offline scenario corpus (CRM and platform-inspired mocks). A finding is a true positive if it correctly identifies a known-injected violation; false positive if it fires on a non-violation; false negative if it misses a known violation. Values are regenerated from `results/derived/scenario_metrics.json`.

TABLE II  
DETECTION PERFORMANCE, OFFLINE MOCK CORPUS (CATEGORY METRICS WITH MICRO-AVERAGE; IMPLEMENTATION VALIDATION UNDER INJECTED LABELS)

| Category               | P            | R            | F1           | TP | FP | FN |
|------------------------|--------------|--------------|--------------|----|----|----|
| Behavioral Divergence  | 1.000        | 1.000        | 1.000        | 6  | 0  | 0  |
| Hidden Endpoint        | 1.000        | 1.000        | 1.000        | 7  | 0  | 0  |
| Policy Violation       | —            | —            | —            | 0  | 0  | 0  |
| Sensitive Field        | 1.000        | 1.000        | 1.000        | 3  | 0  | 0  |
| <b>Overall (micro)</b> | <b>1.000</b> | <b>1.000</b> | <b>1.000</b> | 16 | 0  | 0  |

*Takeaway:* Under injected labels, AIT’s implementation matches the intended DM1–DM3 logic on this controlled corpus.

*Interpretation note:* Perfect controlled scores, when observed, validate that the implementation executes the defined detection logic—not that the system generalizes to unseen or adversarial inputs. External validity requires live platform evaluation (Section VI-E and future work, Section VIII-B).

### D. Platform-Inspired Mock Suite

Beyond the minimal CRM surface used to *isolate* DM1–DM3 (Section VI), we extended the Mock SaaS and Integration Under Test with additional REST paths *inspired by* five major SaaS ecosystems—Slack, GitHub, Google Gmail, Notion, and Trello—each exercising a distinct over-permissioning idiom (hidden history access, private-repository fields under nominally public scope, write calls under read-only mode, content updates under read-only integration assumptions, and mutating write behavior under read-scoped tokens). Two additional integrations are *compliant baselines* included to probe false positives.

**Important framing:** These paths are *synthetic mocks* for controlled evaluation; they are not claims of measurement on vendor production APIs. External validity is addressed in part by the live SaaS evaluation (Section VI-E) and remains a priority for future work (Section VIII-B).

Each scenario is specified as a YAML `TargetConfig` (declared scopes, expected endpoint allowlist, sensitive markers) and executed through the same Coordinator pipeline as the CRM suite. Risk scores follow Eq. (3) with  $(w_H, w_F, w_D) = (25, 20, 15)$ .

TABLE III  
PLATFORM-INSPIRED MOCK RESULTS (RISK SCORES FROM OFFLINE SCENARIO ARTIFACTS; SYNTHETIC MOCKS, NOT VENDOR PRODUCTION APIS)

| # | Platform | Scenario                       | Risk | Sev.   | Det.? |
|---|----------|--------------------------------|------|--------|-------|
| 1 | Slack    | Bot token over-access          | 40   | Medium | Yes   |
| 2 | GitHub   | PAT broad scope                | 60   | High   | Yes   |
| 3 | Google   | Readonly token write attempt   | 40   | Medium | Yes   |
| 4 | Notion   | Read-only integration mutation | 40   | Medium | Yes   |
| 5 | Trello   | Read token card creation       | 40   | Medium | Yes   |
| 6 | Slack    | Compliant bot                  | 0    | N/A    | No    |
| 7 | GitHub   | Compliant app                  | 0    | N/A    | No    |

*Takeaway:* AIT maintains detection on heterogeneous platform-inspired mock idioms while reporting zero risk on the compliant baselines included in the generated table.

**Interpretation:** As with Table II, perfect agreement with declared labels on this deterministic corpus validates end-to-end wiring and reporting across multiple API idioms, not generalization to arbitrary production tenants.

### E. Live SaaS Evaluation

To bridge the gap between controlled mock evaluation and real platform behavior, AIT provides an instrumented live probe runner (`ait/live_runner.py`) for sandbox accounts using read-only tokens from environment variables. Plans under `configs/live/` specify base URL, declared endpoints, and request sequences. Real sandbox Task 6 execution is **BLOCKED** without credentials and is recorded as NOT RUN; no fabricated live observations are substituted.

**GitHub sandbox:** Declared read-only probe design (allowlisted hosts/paths; policy mismatch detection when un-

declared surfaces are reached). Empirical result: NOT RUN (blocked).

**Notion sandbox:** Declared read-path validation design (GET /v1/users/me). Empirical result: NOT RUN (blocked).

TABLE IV

LIVE SAAS EVALUATION RESULTS (SANDBOX ACCOUNTS; UNAVAILABLE ARTIFACTS RENDER AS NOT RUN)

| # | Platform | Scenario             | Violation Type                           | Risk | Model Boundary | Dec.? |
|---|----------|----------------------|------------------------------------------|------|----------------|-------|
| 1 | GitHub   | Live scope boundary  | None                                     | 0    | model_boundary | No FP |
| 2 | Notion   | Read-path validation | None                                     | 0    | model_boundary | No FP |
| 3 | GitHub   | REST smoke probe     | Hidden endpoint: /user/orgs, /user/repos | 50   | model_boundary | Yes   |

*Takeaway:* Live sandbox rows remain unavailable until Task 6 completes with real credentials and redacted artifacts. Design coverage (positive and negative control plans) is in place; empirical claims await those artifacts.

*Interpretation note:* When live results exist, a positive GitHub finding would indicate the supplied token’s actual capabilities exceeded the declared policy—a policy mismatch in the evaluation setup, not a vulnerability in the GitHub platform. No live durations are reported here (NOT RUN).

#### F. Live GitHub REST Smoke Probe (Optional)

Complementing the structured sandbox plans in Section VI-E, a separate read-only probe is defined to exercise `api.github.com` under a stricter policy allowlisting only `/user`, with additional undeclared `/user/repos` and `/user/orgs` requests. Empirical smoke-probe results remain NOT RUN until a hashed live artifact is selected in `configs/paper_artifacts.yaml`. When executed with a read-only PAT via `AIT_GITHUB_TOKEN`, the signal of interest is *policy mismatch*, not server-side exploitability.

#### G. Robustness and Noise

To move beyond purely “clean” traces, the repository includes robustness and evasion scenario YAML (noise, partial exposure, path ambiguity, FN4 field-alias negatives, and model-boundary cases). Table V reports suite status from the selected `robustness_metrics` artifact required by the offline release gate (`robustness_results`). Deterministic reproducibility across five repetitions is recorded in the hashed `reproducibility` artifact.

#### H. Scalability of Analysis

**Isolation note:** This micro-benchmark measures Analysis Engine latency only; full end-to-end Coordinator runs remain dominated by network I/O and OAuth/token work; live SaaS timings in Section VI-E are NOT RUN.

Table VI reports median, p95, and MAD wall time from the hashed offline benchmark artifact (warmups and repetitions recorded in provenance). Risk scores are taken from the same synthetic-log protocol.

*Takeaway:* Detection output is stable as synthetic log width grows within this micro-benchmark; analysis time does not reflect WAN latency.

TABLE V  
ROBUSTNESS / EVASION SUITE STATUS

| Suite / slice                     | Status                        |
|-----------------------------------|-------------------------------|
| in_scope_passed                   | PASS                          |
| in_scope (n=13)                   | PASS                          |
| in_scope micro P/R/F1             | 1.000/1.000/1.000             |
| in_scope Wilson P/R               | [0.676,1.000] / [0.676,1.000] |
| in_scope counts TP/FP/FN/TN       | 8/0/0/44                      |
| model_boundary micro P/R/F1       | FAIL                          |
| model_boundary Wilson P/R         | —/0.000/—                     |
| model_boundary counts TP/FP/FN/TN | [—,—] / [0.000,0.490]         |
| model_boundary Yes                | 0/0/4/12                      |

TABLE VI  
ANALYSIS ENGINE MICRO-BENCHMARK (SYNTHETIC LOG WIDTH; MEDIAN/P95/MAD FROM OFFLINE BENCHMARK\_SUMMARY ARTIFACT)

| Allowlisted paths | Median (ms) | p95 (ms) | MAD (ms) | Risk |
|-------------------|-------------|----------|----------|------|
| 10                | 0.022       | 0.023    | 0.000    | 25   |
| 50                | 0.081       | 0.098    | 0.001    | 25   |
| 100               | 0.158       | 0.175    | 0.001    | 25   |
| 500               | 0.797       | 0.846    | 0.007    | 25   |
| 1000              | 1.562       | 1.827    | 0.020    | 25   |

#### I. Tool Capability Comparison

RESTler and EvoMaster target complementary server-side fault classes. A pinned Docker comparison protocol lives under `comparison/`; until a hashed `tool_comparison` artifact is selected, Table VII reports NOT RUN rather than invented detection outcomes. Do not interpret absence as “no findings.”

TABLE VII  
TOOL CAPABILITY COMPARISON (SELECTED ARTIFACT OR NOT RUN)

| Finding Category         | AIT | RESTler | EvoMaster |
|--------------------------|-----|---------|-----------|
| Hidden Endpoint Access   | —   | NOT RUN | NOT RUN   |
| Sensitive Field Access   | —   | NOT RUN | NOT RUN   |
| Behavioral Divergence    | —   | NOT RUN | NOT RUN   |
| Server-side Input Errors | —   | NOT RUN | NOT RUN   |
| OpenAPI Spec Violations  | —   | NOT RUN | NOT RUN   |

*Takeaway:* When executed, the taxonomy is intended to clarify complementary deployment: server fuzzers for implementation bugs, AIT for integration policy drift. No external-tool run is claimed here.

#### J. Log-Based Replay: Researcher-Constructed Traces

To provide an alignment signal without requiring live platform credentials, we conducted a log-based replay experiment. Public post-mortem disclosures from three documented SaaS security incidents—CircleCI (2023) [4], Okta (2022) [3], and the GitHub OAuth token exposure (2022) [5]—were manually reconstructed into `CapturedExchange` sequences (researcher-constructed traces from public disclosures, not ground-truth API logs). These traces were fed to AIT’s

Analysis Engine and compared against expected categories. Table VIII is regenerated from the hashed replay artifact.

TABLE VIII  
LOG-BASED REPLAY: EXPECTED VS. OBSERVED CATEGORIES  
(RESEARCHER-CONSTRUCTED TRACES)

| Incident                     | Expected categories                                            |
|------------------------------|----------------------------------------------------------------|
| circleci-2023-reconstruction | behavioral_divergence, hidden_endpoint, sensitive_field_access |
| github-2022-reconstruction   | behavioral_divergence, hidden_endpoint, sensitive_field_access |
| okta-2022-reconstruction     | behavioral_divergence, hidden_endpoint, sensitive_field_access |

*Takeaway:* Replay provides *plausibility evidence* that AIT’s detection categories map to independently reported incident classes, not empirical precision/recall on real platform data.

#### K. Risk Score and Sensitivity Analysis

Under the default weights  $(w_H, w_F, w_D) = (25, 20, 15)$ , multi-finding scenarios can enter the Critical band ( $> 75$ ). Table IX reports  $\pm 30\%$  single-weight perturbations for selected CRM scenarios from the hashed sensitivity artifact.

TABLE IX  
RISK SCORE SENSITIVITY ( $\pm 30\%$  WEIGHT VARIATION; SELECTED CRM SCENARIOS FROM OFFLINE ARTIFACTS)

| Scenario                      | Weight Varied | $-30\%$ |
|-------------------------------|---------------|---------|
| crm-s1-hidden-both-phases     | $w_H$         | 17.50   |
| crm-s2-sensitive-allowed-path | $w_F$         | 28      |
| crm-s3-combined               | $w_D$         | 75.50   |
| crm-s3-combined               | $w_H$         | 72.50   |
| crm-s3-combined               | $w_F$         | 68      |

Band transitions, when present, are recorded in `sensitivity_summary`; the score remains a triage signal, not a precise risk measurement.

#### L. Artifact Provenance

Table X lists the selected paper artifacts and SHA-256 prefixes used for this compilation.

TABLE X  
SELECTED ARTIFACT PROVENANCE

| Artifact            | Path                                                                                  |
|---------------------|---------------------------------------------------------------------------------------|
| benchmark_summary   | results/derived/benchmark_summary.json                                                |
| github_readonly     | results/raw/live/github-readonly/github-readonly-20260727T0732518960807-9ca52435.json |
| github_smoke        | results/raw/live/github-smoke/github-smoke-20260727T0732527444412-911e040c.json       |
| notion_readonly     | results/raw/live/notion-readonly/notion-readonly-20260727T0732547920412-8d8507a9.json |
| offline_manifest    | results/derived/offline_manifest.json                                                 |
| replay_match_table  | results/derived/replay_match_table.json                                               |
| reproducibility     | results/derived/reproducibility.json                                                  |
| robustness_metrics  | results/derived/robustness_metrics.json                                               |
| scenario_metrics    | results/derived/scenario_metrics.json                                                 |
| sensitivity_rows    | results/raw/sensitivity/sensitivity_rows.json                                         |
| sensitivity_summary | results/derived/sensitivity_summary.json                                              |
| tool_comparison     | null                                                                                  |

#### M. Discussion

The dual-phase model correctly differentiated baseline behavior (three authorized endpoints) from mutated behavior (four endpoints including one unauthorized). Static audits miss the key distinction here: the billing call was *OAuth-authorized yet policy-violating* under the declared allowlist. The live SaaS evaluation (Table IV) is designed to provide the same detection path on live vendor APIs, but empirical sand-box rows remain NOT RUN/BLOCKED without credentials. Replay and platform mocks (Table III) support that C1-C3 map to realistic abuse shapes without changing core detection rules. AIT’s value lies not in replacing existing tools but in introducing a missing verification layer between declared policy and observed behavior.

#### VII. FALSE NEGATIVE ANALYSIS

A rigorous analysis of cases where AIT will fail to detect a violation is essential to characterize the security guarantees of the framework. Four evasion scenarios are identified:

**FN1 — Out-of-Band Exfiltration.** An integration that accesses authorized endpoints but exfiltrates data via a separate webhook, outbound HTTP request, or side channel will not be detected, as AIT observes only inbound API calls to the instrumented SaaS surface. *Mitigation direction:* network egress monitoring or outbound traffic analysis, out of scope for the current framework.

**FN2 — Shared Token Misuse.** If two integrations share an OAuth token issued to a different client ID, AIT—which attributes calls by run ID, not token identity—will not associate the unauthorized calls with the IUT. *Mitigation direction:* per-integration token isolation at the platform level.

**FN3 — Time-Delayed or Trigger-Based Activation.** An integration that activates unauthorized access after a fixed delay (e.g., 30 days post-installation) or via an external command-and-control signal will evade the two-phase model, which is bounded by the test session duration. *Mitigation direction:* continuous runtime monitoring beyond point-in-time assessment.

**FN4 — Response-Field Aliasing.** If an integration reads sensitive data from a field whose name does not match any token in `sensitive_markers` (e.g., a renamed or obfuscated field in a non-standard API response schema), taint detection will fail. *Mitigation direction:* value-pattern matching in addition to field-name matching; semantic field classification using NLP. The mock harness reproduces this miss explicitly (Section VI-G, FN4 negative row).

These false negative cases are not implementation bugs but fundamental constraints of the black-box, session-bounded, field-name-based detection model. Acknowledging them explicitly bounds the framework’s security guarantees and scopes the threat model accurately.

#### VIII. LIMITATIONS AND FUTURE WORK

##### A. Current Limitations

**External Validity:** Primary evidence comprises mock scenarios, replayed public incidents, and a live probe design for



two sandbox accounts (GitHub and Notion) whose Task 6 empirical rows are NOT RUN/**BLOCKED** without credentials. The framework has not been evaluated tenant-wide on live production SaaS platforms; this remains the primary limitation.

**Evaluation Scale:** The offline mock corpus comprises 16 YAML scenarios: 6 CRM (3 isolation + 3 controls) and 10 platform-inspired (7 violation-oriented + 3 compliant controls). Live sandbox plans exist but empirical runs are blocked. Together they remain a finite corpus and do not explore all edge cases, adversarial inputs, or unexpected API behavior. A larger, more adversarial test suite is needed.

**Scoring Model:** The heuristic design-constant weights are justified and sensitivity-analyzed, but not empirically validated against a ground-truth dataset of real integration violations. Allow/deny live policy checks (and any associated score addends) remain future work / not yet implemented.

**Two-Phase Model:** Real integrations may exhibit variation across more state dimensions than two phases can capture (Section VII, FN3).

**In-Memory State:** Non-persistent and not multi-process safe; a production deployment requires a persistent backend.

**Live Runner Auth:** The current bearer-token auth shape covers GitHub, Notion, Slack, and Google; platforms with non-standard auth conventions (query-param tokens, multi-step flows) require per-platform adapters before evaluation.

## B. Future Work

- 1) *Live platform evaluation* against Slack, GitHub, and Google Workspace sandbox accounts at broader scale—the single highest-impact next step.
- 2) *Automatic policy inference* from OpenAPI specifications and historical access logs.
- 3) *ML-based anomaly detection* for access patterns not requiring pre-specified allowlists.
- 4) *Multi-phase state-machine-guided execution* to cover richer behavioral spaces.
- 5) *Outbound traffic monitoring* to address FN1 (out-of-band exfiltration).
- 6) *Value-pattern taint matching* to address FN4 (response-field aliasing).
- 7) *CI/CD pipeline integration* for automated regression testing on each integration deployment.
- 8) *Empirical risk-weight calibration* against a labeled dataset of real integration violations.

## IX. CONCLUSION

AIT closes a gap left by server-focused fuzzers and static scope tools: black-box reconciliation of observed integration traffic against an explicit policy artifact. The design combines an *integration-centric* test subject, *dual-phase* stimulation for C3, and *instrumentation-free* taint markers for C2. A live SaaS evaluation pipeline (`ait/live_runner.py`) is implemented for instrumented sandbox probes, but GitHub and Notion Task 6 results remain NOT RUN/**BLOCKED** without credentials—no live risk scores are claimed.

Claims are scoped to controlled mocks, live sandbox evaluations, optional smoke probes, and replayed public incidents; false-negative analysis bounds guarantees. Broader tenant-scale validation remains future work. This work establishes *integration-policy conformance* as a first-class security property in SaaS ecosystems. AIT enables organizations to verify integration behavior before and after deployment (Fig. 4), tightening least-privilege practice under zero-trust assumptions.

## ACKNOWLEDGMENT

The authors thank the Department of CSE (IoT & Cybersecurity Including Blockchain Technology), Bangalore Institute of Technology, for supporting this work under Major Project Phase 1 (BIC685).

## REFERENCES

- [1] Cloud Security Alliance, “SaaS Security Survey Report,” 2023.
- [2] CISA, “Alert (AA20-352A): Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations,” Dec. 2020.
- [3] Okta, “Okta Security Incident: January 2022,” Security Advisory, Mar. 2022.
- [4] CircleCI, “CircleCI Security Incident Disclosure and Recommended Customer Actions,” Jan. 2023. [Online]. Available: <https://circleci.com/blog/jan-4-2023-security-alert/>
- [5] GitHub, “Security Alert: Attack Campaign Involving Stolen OAuth User Tokens Issued to Two Third-Party Integrators,” Apr. 2022. [Online]. Available: <https://github.blog/2022-04-15-security-alert-stolen-oauth-user-tokens/>
- [6] F. Aliyu, C. Benson, and M. Ismail, “Over-Privileged OAuth Grants in Enterprise SaaS: Measurement and Risk Characterization,” *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3211–3225, Jul./Aug. 2023.
- [7] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012.
- [8] N. Sakimura, J. Bradley, and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” RFC 7636, IETF, Sep. 2015.
- [9] N. Sakimura et al., “OpenID Connect Core 1.0,” OpenID Foundation, Nov. 2014. [Online]. Available: <https://openid.net/connect/>
- [10] D. Fett, R. Küsters, and G. Schmitz, “The Web SSO Standard OpenID Connect: In-Depth Formal Security Analysis and Security Guidelines,” *Proc. IEEE CSF*, 2017.
- [11] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner, “Android Permissions: User Attention, Comprehension, and Behavior,” *Proc. SOUPS*, 2012.
- [12] OWASP, “OWASP API Security Top 10 – 2023 Edition.” [Online]. Available: <https://owasp.org/www-project-api-security/>
- [13] C. Mainka, V. Mladenov, J. Schwenk, and T. Wich, “SoK: Benchmarking the Effectiveness of OAuth Grant Type Enforcement Across Major SaaS Providers,” *Proc. ACM CCS*, 2022.
- [14] X. De Carné de Carnavalet and P. C. van Oorschot, “Third-Party Application Permission Practices: A Large-Scale Analysis of OAuth over Major Identity Providers,” *Proc. USENIX Security*, 2023.
- [15] J. Boucher, A. Kharraz, and E. Kirda, “Scope Guard: Static Policy Enforcement for SaaS Integration Permission Graphs,” *Proc. NDSS*, 2024.
- [16] E. Lundberg, M. Ekstedt, and R. Lagerström, “Runtime OAuth Scope Enforcement via Transparent API Proxy Middleware,” *Proc. IEEE EuroS&P*, 2024.
- [17] M. Zalewski, “American Fuzzy Lop (AFL),” 2014. [Online]. Available: <https://lcamtuf.coredump.cx/afl/>
- [18] J. Pereyda, “Boofuzz: A Fork and Fix of Sulley,” [Online]. Available: <https://github.com/jtpereyda/boofuzz>
- [19] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API Fuzzing,” *Proc. ICSE*, 2019.
- [20] A. Arcuri, “RESTful API Automated Test Case Generation with EvoMaster,” *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 1, 2019.

- [21] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Checking Security Properties of Cloud Service REST APIs," *Proc. IEEE S&P*, 2020.
- [22] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and Understanding Bugs in C Compilers," *Proc. PLDI*, 2011.
- [23] T. Petsios, J. Zhao, A. D. Keromytis, and S. Jana, "SlowFuzz: Automated Domain-Independent Detection of Algorithmic Complexity Vulnerabilities," *Proc. CCS*, 2017.
- [24] S. Kim, Y. Lee, and J. Choi, "DiffMicro: Differential Testing for Semantic Consistency in Microservice API Evolution," *Proc. IEEE ICWS*, 2022.
- [25] V. B. Livshits and M. S. Lam, "Finding Security Vulnerabilities in Java Applications with Static Analysis," *Proc. USENIX Security*, 2005.
- [26] J. Clause, W. Li, and A. Orso, "Dytan: A Generic Dynamic Taint Analysis Framework," *Proc. ISSTA*, 2007.
- [27] W. Enck et al., "TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones," *ACM Trans. Comput. Syst.*, vol. 32, no. 2, 2014.
- [28] F. Benamara, N. Mager, and M. Dacier, "DPIFuzz: A Differential Fuzzing Framework to Detect DPI Elusion Strategies for QUIC," *Proc. ACSAC*, 2020.
- [29] A. Bulekov, B. Das, S. Hajnoczi, and M. Egele, "Fuzztruction: Using Fault Injection to Hack Fuzzing Mutators," *Proc. USENIX Security*, 2023.
- [30] X. Pan, Y. Zhang, and M. Grace, "SynthAPI: Privacy-Preserving Synthetic Data Generation for API Security Testing," *Proc. ACM CODASPY*, 2022.
- [31] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020.
- [32] Y. Ding, R. Phan, and T. Pöppelmann, "Continuous Authorization for Third-Party SaaS Integrations under Zero Trust," *Proc. IEEE CloudCom*, 2023.
- [33] D. Wormke et al., "A Large Scale Investigation of Obfuscation Use in Google Play," *Proc. USENIX Security*, 2022.
- [34] R. Chandramouli, "SaaS Security Posture Management: Taxonomy, Challenges, and Research Directions," NIST Internal Report 8467, 2023.
- [35] FIRST.Org, "Common Vulnerability Scoring System v3.1: Specification Document," 2019. [Online]. Available: <https://www.first.org/cvss/specification-document>